

LECTURE 21

Deadlock Detection, Recovery, and Case Study

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Building a wait-for graph from a Resource-Allocation Graph and detecting deadlock as a cycle, for the single-instance-per-resource-type case
- The multi-instance Detection Algorithm — Available, Allocation, and Request matrices — and how it mirrors the Banker's Algorithm's safety check
- The real trade-off in when to run the detection algorithm: overhead vs. how long deadlocked processes sit stuck
- Recovery by process termination — abort-all vs. abort-one-at-a-time, and the cost factors that decide which process to pick
- Recovery by resource preemption — victim selection, rollback, and the starvation risk
- Case Study: how Windows handles synchronization without any system-wide deadlock detection at all

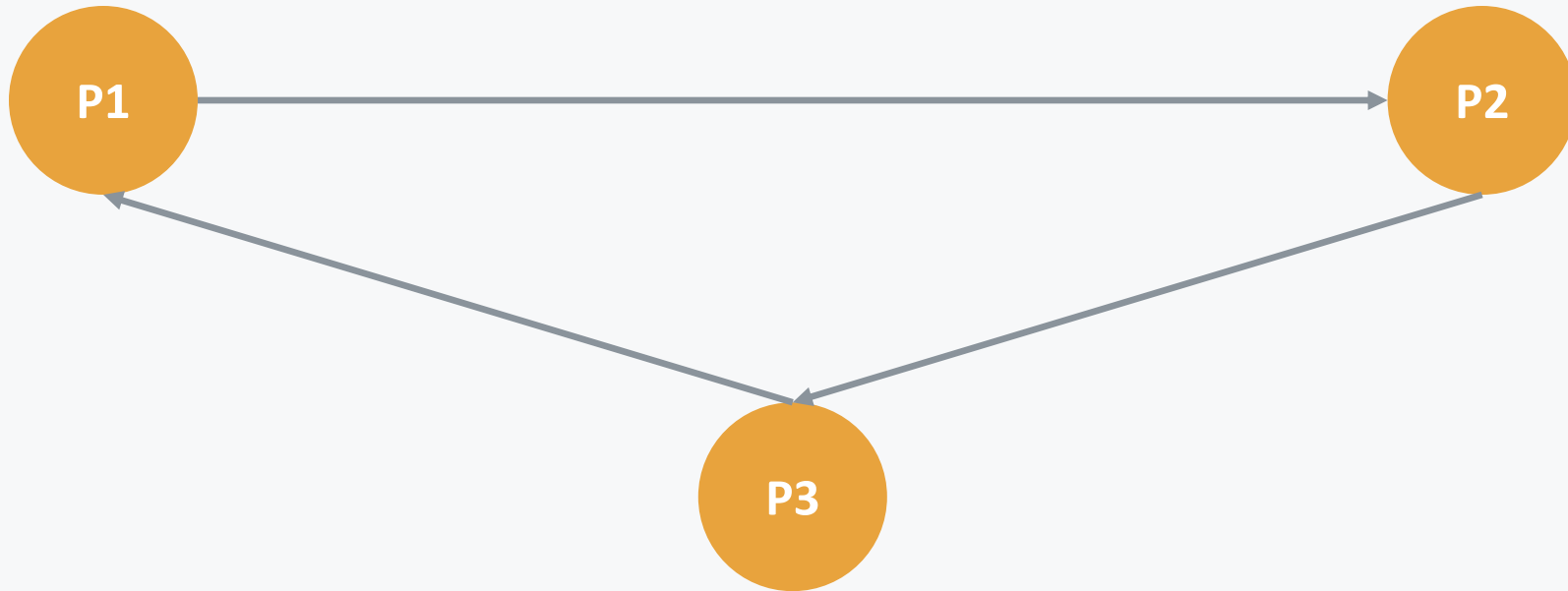
Why Detection and Recovery, Not Just Avoidance?

- Avoidance and prevention work by refusing to let a dangerous state ever occur
- Detection and recovery let the system run freely — no extra bookkeeping on every resource request — and only pay a cost if and when a deadlock actually forms
- This trades a constant, guaranteed overhead (checking every request) for an occasional, larger one (periodically checking for cycles and cleaning up)
- For a system where deadlocks are rare, that trade is often the right one

This approach assumes the OS allows unsafe states — If a system runs with no avoidance or prevention algorithm at all, unsafe states — and therefore deadlocks — are allowed to occur. Detection and recovery exist specifically for that situation.

Single Instance: From RAG to Wait-For Graph

- A wait-for graph removes the resource nodes from a Resource-Allocation Graph entirely
- Draw a direct edge $P_i \rightarrow P_j$ whenever P_i is waiting for a resource currently held by P_j
- With exactly one instance per resource type, this collapse is always unambiguous



Each process waits for a resource held by the next — $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ closes a cycle

Detecting the Cycle

- The OS periodically builds the wait-for graph from its bookkeeping of who holds what and who is waiting for what
- **Cycle detection:** a depth-first search works, costing roughly $O(n^2)$ edge checks for n processes
- No cycle $\rightarrow$ no deadlock, every process can eventually finish
- Any cycle found $\rightarrow$ every process on that cycle is deadlocked

A cycle is both necessary and sufficient here — This clean equivalence — deadlock if and only if a cycle exists — only holds because each resource type has a single instance. Once a resource type can have multiple instances, a cycle no longer guarantees deadlock, because a process on the cycle might proceed using a different instance of the same type. That's exactly why multiple instances need a different, more careful algorithm — covered next.

Multiple Instances: Available, Allocation, Request

- With multiple instances per resource type, the detection algorithm reuses almost exactly the same data-structure shape as the Banker's Algorithm's safety check

Structure	Shape	Meaning
Available	vector, length m	Instances of each resource type currently free
Allocation	matrix, $n \times m$	Instances of each resource type currently held by each process
Request	matrix, $n \times m$	Instances of each resource type each process is currently requesting

The one real difference — The Banker's Algorithm needed Max (a process's declared maximum future demand). The Detection Algorithm has no such promise — Request records only what a process is actually, currently blocked waiting for, right now. Everything else — Work, $\text{Request}(i) \leq \text{Work}$, releasing $\text{Allocation}(i)$ back into Work — is the identical mechanical idea.

The Detection Algorithm

1

Initialize — $Work = Available$. $Finish[i] = false$ for every process i — unless $Allocation(i)$ is all zeros, then $Finish[i] = true$ immediately.

2

Find — An index i such that $Finish[i] = false$ and $Request(i) \leq Work$ (component-wise). If none exists, go to step 4.

3

Simulate completion — $Work = Work + Allocation(i)$, $Finish[i] = true$ — this process can finish and release everything it holds. Go back to step 2.

4

Stop — If $Finish[i] = false$ for some process i , the system is deadlocked — every such process is deadlocked.

Worked Example: Allocation and Request

- Three resource types — A, B, C — with four processes, P0–P3. Available = (0, 0, 0).

Process	Allocation (A, B, C)	Request (A, B, C)
P0	(1, 0, 2)	(0, 0, 0)
P1	(2, 1, 1)	(4, 0, 0)
P2	(2, 1, 2)	(1, 0, 1)
P3	(1, 1, 0)	(0, 2, 0)

Sanity check — Column totals of Allocation are (6, 3, 5); add Available = (0,0,0) and the system's total resource counts are A = 6, B = 3, C = 5 — exactly accounted for.

Worked Example: Running the Algorithm

- Initialize. Work = (0,0,0). All Finish[P0..P3] = false.
- Round 1. P0: $(0,0,0) \leq (0,0,0)$ ✓ — select P0. Work = $(0,0,0) + (1,0,2) = (1,0,2)$. Finish[P0]=true.
- Round 2. Work=(1,0,2). P1: $(4,0,0) \leq (1,0,2)$? $4 > 1$ — no. P2: $(1,0,1) \leq (1,0,2)$? ✓ — select P2. Work= $(1,0,2) + (2,1,2) = (3,1,4)$. Finish[P2]=true.
- Round 3. Work=(3,1,4). P1: $4 > 3$ — no. P3: $2 > 1$ on B — no. No process found. Stop.

P0, P2 — Finish = true

Requests satisfied in order P0 → P2; not deadlocked

P1, P3 — Finish = false

Requests can never be satisfied with the remaining Work; deadlocked

When and How Often to Run Detection

- **Run it on a fixed schedule** — e.g., once every N minutes. Short interval catches deadlocks quickly but spends more CPU; long interval wastes less CPU but lets deadlocked processes sit frozen longer.
- **Run it when there's a symptom** — most commonly, when CPU utilization drops below some threshold (say, 40%), a cheap, indirect signal that processes may be blocked waiting on each other.

There's no universally correct interval — The right choice depends on how expensive a deadlocked process is to leave stuck, against how expensive the detection algorithm itself is on that system — a genuine systems-engineering trade-off, not a fixed rule.

Recovery by Process Termination

- **Abort all deadlocked processes at once** — simple and immediate, but wasteful since some processes might have been one step from finishing.
- **Abort one process at a time** — re-running detection after each kill, stopping as soon as the cycle breaks. Less damage, more overhead.

Victim-selection cost factors (one-at-a-time approach):

- **Priority** — low-priority processes are generally cheaper to sacrifice
- **Runtime so far / remaining** — a process close to finishing has lost much more if killed
- **Resources already held** — killing a process holding many resources frees more, at greater cost
- **Resources still needed** — a process needing very few more might be close to finishing on its own

Recovery by Resource Preemption

- **1. Select a victim resource** — using a similar cost function — which resource's removal costs the least to undo.
- **2. Roll back the affected process** — to the most recent safe state it was in, rather than restarted from scratch.
- **3. Prevent starvation** — cap how many times any one process can be picked as a victim, so it can't be selected forever.

Rollback is only cheap if the system already tracks safe checkpoints — Without that bookkeeping, 'rollback' degenerates into 'restart from the beginning,' exactly as wasteful as the abort-all strategy.

Case Study: Deadlock Handling in Windows

- **Dispatcher objects** — mutexes, semaphores, and events that threads wait on via `WaitForSingleObject` / `WaitForMultipleObjects`. Windows never checks whether usage can form a cycle.
- **Critical sections** — a lighter-weight, user-mode mutual-exclusion primitive for threads within a single process.
- **Timeout parameters on wait calls** — a thread can pass a timeout instead of waiting forever; this doesn't detect deadlock formally, but stops a careless design from freezing permanently.

A reasonable design choice, not a shortcut — System-wide detection has real runtime cost; avoidance requires processes to declare maximum needs up front — information general-purpose applications essentially never provide.

Key Takeaways

- With a single instance per resource type, collapse the RAG into a wait-for graph — a cycle is necessary and sufficient proof of deadlock
- With multiple instances, the Detection Algorithm uses Available, Allocation, and Request matrices, running exactly like the Banker's safety check, with Request standing in for Need
- When to run detection trades overhead against how long deadlocked processes sit stuck — fixed interval or CPU-utilization threshold
- Recovery by termination: abort all (simple, wasteful) or one at a time with re-detection (less wasteful, more overhead)
- Recovery by resource preemption selects a victim resource and rolls back to a safe state, bounding victim selection to avoid starvation
- Windows runs no system-wide deadlock detection or avoidance — it provides synchronization primitives and wait timeouts, leaving avoidance to the application developer